scx_slo: SLO-Aware CPU Scheduling for Co-located Datacenter Workloads with sched_ext

Thrasos Sazeides `tsazei01@ucy.ac.cy` Markos Yiakoumis `mgiako02@ucy.ac.cy`
Department of Computer Science, University of Cyprus · November 8, 2026 · `github.com/vcTaz/scx_slo`

1. Summary

The proposal planned baseline measurements and a first working scx_slo for week 9. **We could not run any experiment on CloudLab**: the bare-metal nodes our profile needs (`xl170`, with `c220g5` as the alternative) were not available to us, so the testbed was never instantiated. There are therefore **no measured results in this report**. We used the time to build and test everything that does not need the testbed: the feedback controller, the latency windows, a closed-loop simulator, a first partial-mode scx_slo scheduler, and a fully scripted testbed and benchmark harness that runs end to end once nodes are available. In simulation the controller meets the proposal's success criterion; this has yet to be confirmed on hardware.

2. Progress Since the Proposal

1. **Feedback controller** (`crates/slo-ctrl`). Each 100 ms tick it reads p99 over a short (0.3 s) and a long (2 s) window and sets two levers: *k*, the number of cores batch tasks may use, and their time slice. It cuts fast (halve *k* and slice when short-window p99 exceeds 90% of the SLO), holds for 0.5 s, then recovers slowly (one core, then 2 ms of slice, every 0.3 s while long-window p99 is under 70% of the SLO). Missing data means hold.
2. **Latency windows.** Log-linear histograms (8 sub-buckets per power of two, at most 12.5% overestimate of p99) kept as running sums, so a window update costs O(1) and the same code can read a BPF histogram map.
3. **Tests.** 26 unit and synthetic-trace tests (load step, burst, slow drift, missing data, hold-down, limits), all passing.
4. **Simulator** (`crates/slo-sim`). Runs the real controller against a model of memcached (one M/M/1 queue whose service rate drops as batch tasks get more cores and longer slices) and reproduces our planned baselines (Section 3).
5. **scx_slo scheduler, first version** (`scheduler/`). Built on scx_rustland_core (scx v1.1.4). It runs in partial mode, so only batch tasks started through a small `ext_exec` launcher are scheduled by sched_ext while memcached stays on EEVDF. It confines batch tasks to a CPU set (`--cpus`) with a fixed slice, and logs dispatch statistics every second. The controller is not yet wired in, and the scheduler has not been run, because it needs a kernel with sched_ext.
6. **Testbed and harness.** A CloudLab profile (2 nodes, Ubuntu 24.04, 100 GB blockstore), setup scripts for both nodes (sched_ext kernel, scx, memtier_benchmark 2.5.2, batch inputs), a pre-flight check, and benchmark scripts for every baseline: memcached alone, EEVDF, a static cgroup (`AllowedCPUs`), stock scx_rustland, scx_slo on all CPUs, and scx_slo with a fixed core limit. One command runs a load sweep and a parser summarises the results.

3. Simulation Results

Because the testbed was unavailable, we evaluated the controller in simulation with a placeholder SLO of 500 µs on a 16-core machine (load as a percentage of memcached's peak; 5 runs per point). **These numbers come from a model, not from measurements**; only the shape of the trade-off is meaningful.

Variant (simulated)	p99 at 30%	p99 at 50%	p99 at 70%	p99 at 90%	Within SLO, 10–70%	Batch thr., 10–70%
(b) EEVDF, no control	328 µs	655 µs	92 ms	94 ms	4 of 7 loads	100%
(c) Static limit, sized for 90%	147 µs	180 µs	262 µs	459 µs	7 of 7	12%
Fixed half of the cores	180 µs	262 µs	459 µs	2.4 ms	7 of 7	50%
scx_slo (adaptive)	328 µs	387 µs	393 µs	393 µs	**7 of 7**	**91%**

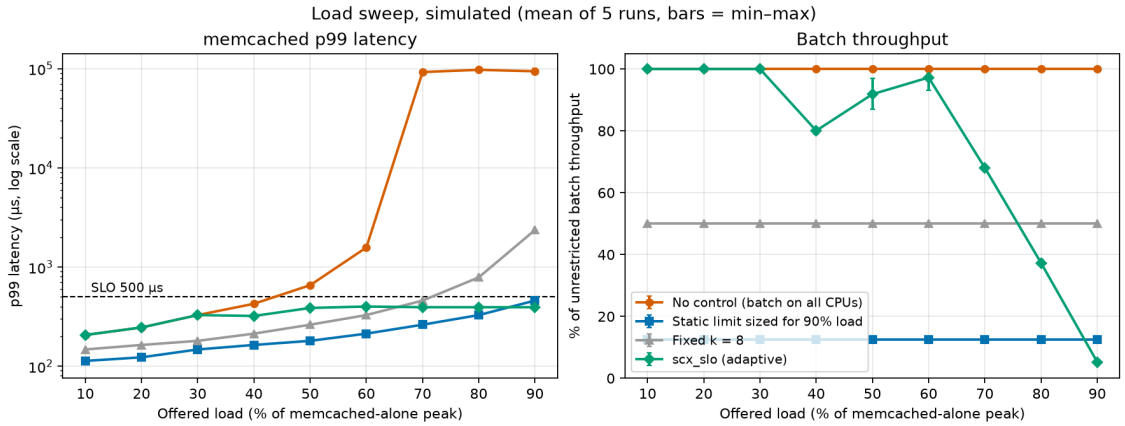


Figure 1. Simulated load sweep. Left: memcached p99 (log scale) against the 500 µs SLO. Right: batch throughput relative to running unrestricted. scx_slo stays under the SLO at every load while giving batch jobs most of the machine up to 60% load.

In a load burst (30% to 85% for 40 s), scx_slo exceeded the SLO for 3 ticks (0.3 s) and was back under it within 0.5 s, while EEVDF stayed over it for the whole burst. The simulation also exposed four tuning problems that we will address on hardware:

- The first tick of a burst is always lost, because the controller acts one tick after it observes.
- The hysteresis band stops recovery early: at 40% load batch jobs settle at 80% throughput instead of 100%.
- It over-cuts at high load: at 90% load batch throughput falls to 5%, below the static limit's 12%.
- At 50% load the controller oscillates between cutting and recovering.

4. Problems and Risks

- **Testbed (blocking).** No CloudLab nodes were available, so no baseline is measured. The scripts are written and checked, but have never run on real nodes and will likely need small fixes.
- **The SLO is a placeholder.** It will be set from the measured p99 of memcached alone, and the simulation re-run with it.
- **EEVDF may already protect memcached.** In partial mode the fair class runs ahead of sched_ext, so memcached may already be protected without any controller. The first hardware test compares scx_slo on all cores with memcached alone to find out.
- **Latency probe.** The BPF probe that measures memcached's p99 in the kernel is not built yet. Fallback: the load generator reports p99 to the scheduler over a socket.

5. Revised Timeline

The proposal's weeks 7–9 slipped by about one week because of the testbed. Simulation results already de-risk the controller design, so we compress the remaining work:

Weeks	Milestone
10 (9–15 Nov)	Testbed running (or fallback machine); baselines (a)–(d) measured; real SLO chosen; latency probe working, or switch to the socket fallback by 12 Nov
11 (16–22 Nov)	Controller wired into scx_slo through a BPF map (core set and slice); first closed-loop runs on hardware
12 (23–29 Nov)	Tuning based on the four simulation findings; full load sweeps with 5 repetitions; burst reaction time and scheduler overhead
13 (30 Nov–4 Dec)	**Final presentation**

6. Team Contributions

As planned in the proposal, we share every part of the project equally: both of us wrote code for the controller, simulator and testbed scripts, each reviewed the other's changes, and we co-wrote this report.

References

[1] sched_ext schedulers and tools. https://github.com/sched-ext/scx
[2] scx_slo source code, simulation data and scripts. https://github.com/vcTaz/scx_slo